

Note → you can create array at 2 types

① let arr = [1, 2, 3, 4, 5];

② let arr = new Array("Saurabh", "is", "great");

Arrays

An array is a special type of object used to store multiple values in a single variable, accessed by index.

```
const fruits = ["Apple", "Banana", "Mango"];
```

↳ // creating elements
console.log(fruits); // log all elements

```
console.log(fruits[0]); // Accessing array elements
```

```
fruits[1] = "Orange"; // updating elements
```

```
fruits.push("Grape");  
fruits.unshift("Kiwi");
```

// adding elements

```
fruits.pop();  
fruits.shift();
```

// removing elements

Looping through an array:

```
for (let i = 0; i < fruits.length; i++) {  
    console.log(fruits[i]);  
}
```

→ for loop

```
for (let fruit of fruits) {  
    console.log(fruit);  
}
```

→ for...of
loop method

• ~~length~~ → is used to find length of array. (27)

```
fruits.forEach((fruit) => {  
  console.log(fruit)  
});
```

 → forEach() method.

Arrays Method → These methods make working with ~~range~~ arrays easier & cleaner

1. forEach() → executes a function for each element in an array.

Note - (It does not return anything)

Ex -

```
const numbers = [1, 2, 3];  
num.forEach((num) => console.log(num * 2));
```

// Output - 2, 4, 6

2. map() → creates a new array by applying a function to each element.

Ex -

```
const numbers = [1, 2, 3];  
const squared = numbers.map((num) =>  
  num * num);
```

```
console.log(numbers);  
console.log(squared);
```

// output [1, 2, 3]
 [1, 4, 9]

3. filter () → creates a new elements that pass a condition.

Ex -

```
const numbers = [1, 2, 3, 4, 5];  
const even = numbers.filter (x) => x%2 === 0;  
console.log(even); // [2, 4]
```

↳ used to select certain elements from an array

4. Reduce () → reduces the array to a single value by applying a function repeatedly.

Ex -

```
const numbers = [1, 2, 3, 4];  
const sum = numbers.reduce(  
  (acc, num) => acc + num, 0);  
console.log(sum); // output → 10
```

Note, Used for cumulative calculations (sum, product, etc.)

And much more methods - like →

5) Shift	6) unshift	7) Slice	8) Sort	9) indexOf
10) find	11) push	12) pop	13) splice	

~~ES6~~ ES6+ Features (Modern JS)

Template Literals : Def :

They are a new way to work with strings. They use backticks (`) instead of quotes and allow embedding variables or expressions easily.

Syntax -

string text `\${expression}`

Ex -

```
let name = "Saurabh"
```

```
let age = 25;
```

```
console.log("My name is ${name}, &  
I am ${age} years old.");
```

// output = My name is Saurabh, &
I am 25 years old.

Advantages →

- Easier to combine text + variables
- can span multiple lines

Destructuring : Def :

It allows to unpack values from arrays or objects & assign them to variables easily.

* Array Destructing

```
let fruits = ["apple", "banana", "mango"]  
let [first, second, third] = fruits;  
console.log(first);  
console.log(second);
```

* Object Destructing

```
let user = { name: "Saurabh",  
             age: "22",  
             city: "Prayagraj"  
};
```

```
let { name, age } = user;  
console.log(name); // Saurabh  
console.log(age);  // 22
```

```
let { city: location } = user; Remaining  
console.log(location); variables.  
// output → Prayagraj
```

```
[ let { country = "India" } = user; ] Default  
values.
```

XX Spread & Rest operators :-

Both uses the same syntax (...) but behave differently depending on where they're used.

(*) Spread operator → used to expand the elements of an array or object.

Ex 1 - Arrays -

```
let arr1 = [1, 2, 3];
let arr2 = [...arr1, 4, 5];
console.log(arr2);
// [1, 2, 3, 4, 5]
```

Ex 2 = Objects -

```
let person = { name: "Saurabh", age: 25 };
let details = { ...person, city: "Lucknow" };
console.log(details);
//   name
```

(*) Rest operator → used to collect remaining values into a single variable.

Ex - function

```
function sum (... numbers) {
  return numbers.reduce((total, n) =>
    {
      total + n, 0
    }
  );
}
```

```
console.log (sum (1, 2, 3, 4)); // 10
```

Ex - Objects / Arrays -

```
let { name, ... rest } = { name: "Saurabh",
  age: 23, id: 1 };
console.log (rest);
```

Modules (Import / exports) : Def:

Modules allows to split your code into multiple files and reuse functions, variables, or classes by exporting & importing them.

* Exporting
from file math.js :

```
export const add = (a, b) => a + b;
export const sub = (a, b) => a - b;
```

Or export all at once:

```
const multiply = (a, b) => a * b;  
export default multiply;
```

(*) Importing

In another file index.js:

```
import { add, sub } from './math.js';  
import multiply from './math.js';
```

```
console.log(add(5, 3)); // 8  
console.log(multiply(5, 3)); // 15
```

↳ Note

↳ Cleaner & modular code

↳ easy to maintain & reuse function.